

Reinforcement learning for control of an inverted pendulum robot simulation in an openAI gym environment

Sarthak Mehulbhai Desai

INTRODUCTION

Inverted pendulum abstract many real-world systems that require vertical stabilization : The human posture, Segway robots, vectored thrust rockets etc. Cart-Pole systems are a class of inverted pendulum systems. In this project a custom simulation environment, modelled on a “self-balancing” inverted pendulum robot is created in OpenAI’s Gym toolkit. The performance of four reinforcement learning (RL) algorithms, two policy based: PPO - Proximal Policy Optimization and A2C - Synchronous advantage actor critic, and two Q-learning based: DQN - Deep Q network and QR-DQN - Quantile Regression DQN, is tested on the custom environment.

MODEL AND TASK DEFINITION

The robot model is based on modified cart-pole state space equations.

$$\ddot{x} = \frac{F + m_p \cdot l(\dot{\theta}^2 \sin \theta - \ddot{\theta} \cos \theta)}{m_c + m_p} \quad g = -9.8 \frac{m}{s^2}, \text{acceleration due to gravity}$$
$$\ddot{\theta} = \frac{g \sin \theta + \cos \theta \left(\frac{-F - m_p l \dot{\theta}^2 \sin \theta}{m_c + m_p} \right)}{l \left(\frac{4}{3} - \frac{m_p \cos^2 \theta}{m_c + m_p} \right)} \quad m_c = 642 \text{ gm}, \text{mass of robot base}$$
$$\quad m_p = 113 \text{ gm}, \text{mass of robot body}$$
$$\quad l = 44 \text{ cm}, \text{half length of robot from wheel axel}$$
$$\quad F = 4.75 \text{ N}, \text{force applied to centre of gravity}$$

Thus, the model has 4 states: Position - x , Velocity- $\dot{x}$, Angle with the vertical to ground - θ , Angular velocity about axel passing through both wheels - $\dot{\theta}$. And it can take two actions, either move left or right with the force applied by its motors, given as $\tau = r \times F$. As rated torque of robot motor is 7.83 N cm, and wheel diameter is 6.6 cm, the force is given as $F = 2 \times 2.373$. ($2 \times$ – because robot has 2 motors). The model is implemented in openAI gym by creating a custom environment, compliant with the toolkit’s class specifications. OpenAI’s Stable-Baseline3 implementations were used to create models for each of the RL algorithms to be tested.

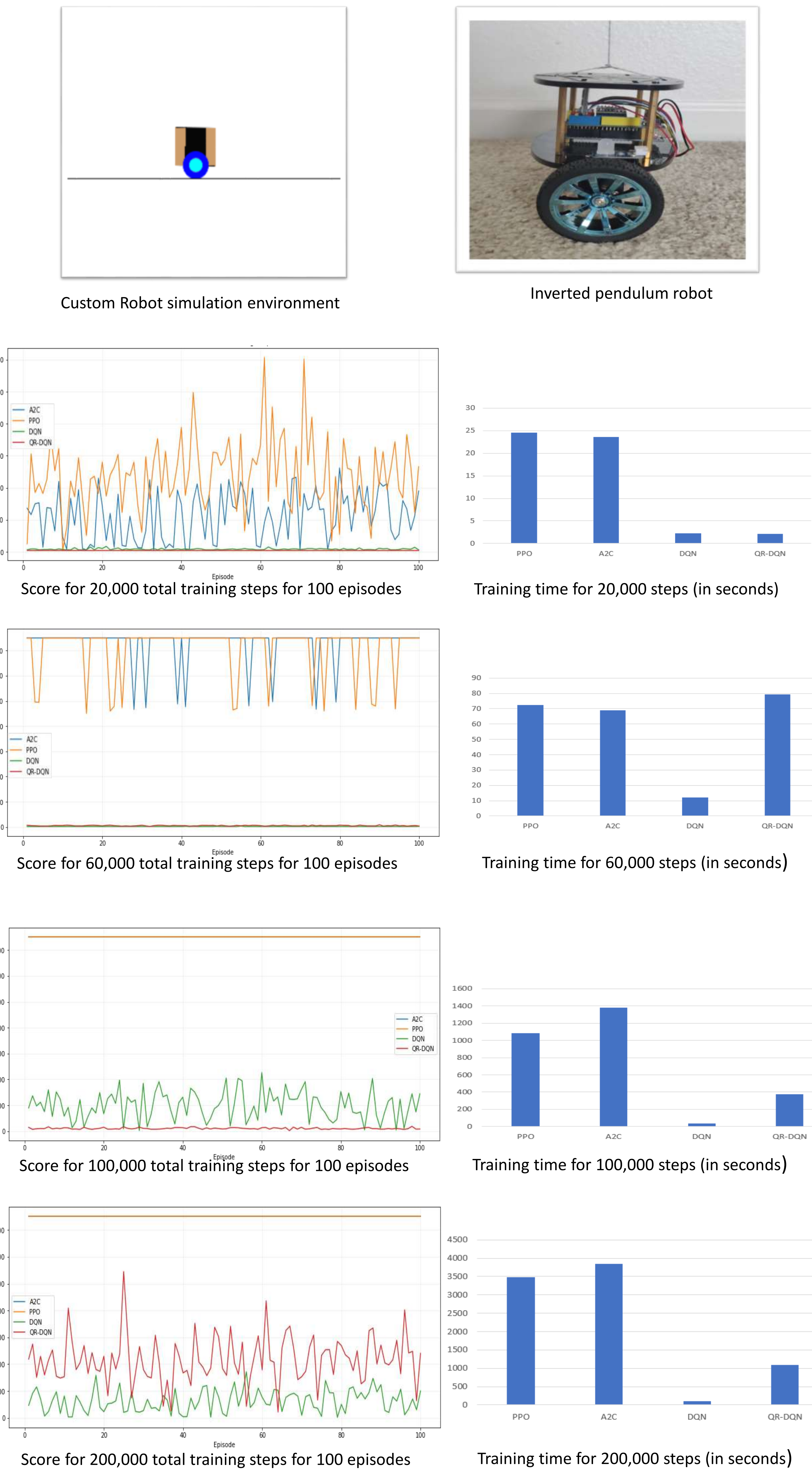
ALGORITHM DEFINITION

- Two policy optimization based reinforcement learning algorithms were explored in this project:
1. A2C - Advantage actor critic is an on-policy, policy gradient algorithm. It learns the value function for one policy while following it. The advantage function can be approximated as : $A(s_t, a_t) = r_{t+1} + \gamma V_v(S_{t+1}) - V_v(s_t)$. The model learns by optimizing for critic loss and actor loss. The critic basically tries to apply MSE on the advantage function. For the actor (in the case of categorical actions), a categorical distribution is sampled then the loss is calculated using the negative log likelihood, scaled by the advantage
 2. PPO - The Proximal Policy Optimization algorithm combines ideas from A2C (having multiple workers), and it uses a trust region to improve the actor.

Two Q-learning based reinforcement learning algorithms were explored in this project:

For Q-learning, a Q-value ($Q(s, a)$), is the expected reward for each state-action pair. The goal of the agent is to learn a policy that maximizes the reward it receives over a long period.

1. DQN - Deep Q network RL, aims to train a policy that tries to maximize the discounted, cumulative reward. This project uses the stable-baseline3’s default deep network architecture to perform deep learning for this purpose.
2. QRDQN - Quantile Regression DQN builds on Deep Q-Network (DQN) and make use of quantile regression to explicitly model the distribution over returns, instead of predicting the mean return.



SUMMARY

The objective of the robot model is to stand upright. hence, an episode terminates if any one of the following occurs: Robots angle is greater than $\pm 10^\circ$, Robot Position is greater than ± 1 m from the center of the display, Episode length is greater than 1500. Episode length of 1500 corresponds to 5 minutes in real time. This because the sampling of state is done every 0.02 seconds. A reward of +1 is given for each step the robot doesn’t fall and thus doesn’t terminate the episode. A Score is defined as the sum of all rewards in an episode. To train the models an MLP policy is employed with an architecture of four layers each of 128 neuron width and tanh activation function for A2C and PPO. And the default MLP architecture of stable-baseline3 for DQN and QR-DQN. While training a score of 6000 (2 minutes in real time) is kept as a threshold to stop a training episode. While the testing is done for the maximum possible - 15000 steps. At lower than 10,000 number of total training steps, none of the models perform well. With A2C and PPO reaching a maximum score of 175 and 24 respectively, and DQN and QR-DQN averaging at 5. With 20,000 total training steps PPO achieves higher average score than A2C. While DQN and QR-DQN continue to score less than 15. Between 20,000 and 60,000 total training steps, A2C starts to outperform PPO achieving higher average scores. A2C and PPO can reach the maximum possible reward – score of 1500 in every episode with 70,000 total training steps and higher. DQN starts to achieve an average scores higher than 400, only after 100,000 total training steps. However, the performance of QR-DQN does not improve. After 200,000 total training steps QR-DQN’s performance increases sharply, and it outperforms DQN.

CONCLUSIONS

In this project a custom openAI gym environment was created, simulating an inverted pendulum robot. Four RL algorithms, two policy gradient based and two Q learning based, were used to train the model. Both the policy gradient based models : PPO and A2C performed better, achieving the perfect score of 15000 in less than 60,000 total training steps. Though the learning rate of PPO model is higher initially, A2C trained model was found to achieve perfect score sooner i.e. with fewer total training steps. Thus, if the physical robot is required to be trained to stand upright using transfer model based learning. A model trained in simulation environment with A2C RL algorithm, would be the best choice among the four tested RL.

BIBLIOGRAPHY

- Mnih, Volodymyr, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy P. Lillicrap, Tim Harley, David Silver and Koray Kavukcuoglu. “Asynchronous Methods for Deep Reinforcement Learning.” ICML (2016).
- Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford and Oleg Klimov. “Proximal Policy Optimization Algorithms.” ArXiv abs/1707.06347 (2017).
- Özalp, R., Varol, N.K., Taşci, B., Uçar, A. (2020). A Review of Deep Reinforcement Learning Algorithms and Comparative Results on Inverted Pendulum System. In: Tsihrintzis, G., Jain, L. (eds) Machine Learning Paradigms. Learning and Analytics in Intelligent Systems, vol 18. Springer, Cham. https://doi.org/10.1007/978-3-030-49724-8_10
- Dabney, Will, Mark Rowland, Marc G. Bellemare and Rémi Munos. “Distributional Reinforcement Learning with Quantile Regression.” AAAI (2018).